

## بازی کاشی کاری

بارچین و چاروس در حال بازی روی جدولی  $2N \times 2M$  از کاشی‌ها هستند. ردیف‌ها از بالا به پایین از 0 تا  $2N - 1$  شماره‌گذاری شده‌اند، و ستون‌ها از چپ به راست از 0 تا  $2M - 1$  شماره‌گذاری شده‌اند. برای  $0 \leq i < 2N$  و  $0 \leq j < 2M$ ، خانه‌ی موجود در سطر  $i$  و ستون  $j$  را با  $(i, j)$  نشان می‌دهیم.

بارچین به چاروس  $N \cdot M$  **بلوک**، یکی پس از دیگری می‌دهد. هر بلوک یک مربع  $2 \times 2$  است که از چهار کاشی  $1 \times 1$  تشکیل شده است. بارچین کاشی‌های هر بلوک را سیاه یا سفید رنگ کرده است و تضمین می‌کند که **حداقل یکی از کاشی‌ها سفید باشد**.

چاروس باید هر بلوک را **بلافاصله پس از دریافت آن** روی جدول قرار دهد، بدون اینکه بداند بعداً چه بلوک‌هایی دریافت خواهد کرد. بلوک‌ها قابل چرخش نیستند. هر بلوک باید کاملاً درون جدول قرار گیرد و دقیقاً چهار خانه‌ی جدول را بپوشاند. علاوه بر این، کاشی بالا سمت چپ هر بلوک باید خانه‌ای را بپوشاند که مختصات سطر و ستون آن زوج باشد. هر خانه از جدول می‌تواند حداکثر توسط یک بلوک پوشش داده شود.

بارچین در صورتی برنده‌ی بازی است که پس از قرار دادن هر بلوک، یک مربع  $2 \times 2$  از خانه‌ها وجود داشته باشد که توسط چهار کاشی سیاه پوشانده شده باشد. به بیانی دیگر، اگر خانه‌های  $(a, b)$ ،  $(a + 1, b)$ ،  $(a, b + 1)$ ،  $(a + 1, b + 1)$  همگی توسط کاشی‌های سیاه به شرط  $0 \leq a < 2N - 1$  و  $0 \leq b < 2M - 1$  پوشانده شده باشند، بارچین برنده می‌شود. نیازی نیست مقادیر  $a$  و  $b$  زوج باشند.

چاروس در صورتی برنده می‌شود که تمام  $N \cdot M$  بلوک را بدون اینکه بارچین برنده شود، قرار دهد. توجه داشته باشید که قرار دادن  $N \cdot M$  بلوک، جدول را به طور کامل پوشش می‌دهد.

وظیفه‌ی شما این است که یک استراتژی برای چاروس پیاده کنید تا در بازی برنده شود. می‌توان ثابت کرد که تحت محدودیت‌های داده شده، چاروس با صرف نظر از رنگ بلوک‌هایی که بعداً دریافت می‌کند همیشه می‌تواند بلوک‌ها را طوری قرار دهد که برد را تضمین کند.

## جزئیات پیاده‌سازی

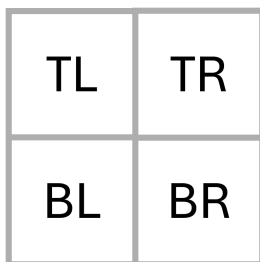
شما باید دو تابع را پیاده‌سازی کنید:

```
void init(int N, int M)
```

- $N$  : نصف تعداد ردیف‌های جدول.
- $M$  : نصف تعداد ستون‌های جدول.
- این تابع دقیقاً یک بار به ازای هر نمونه، در ابتدای اجرای برنامه‌ی شما فراخوانی می‌شود.

```
std::pair<int, int> receive_block(int TL, int TR, int BL, int BR)
```

- $BR, BL, TR, TL$  : به ترتیب رنگ کاشی‌های بالا چپ، بالا راست، پایین چپ و پایین راست بلوک فعلی، همانطور که در شکل زیر نشان داده شده است. هر مقدار 0 (سفید) و یا 1 (سیاه) است.
- این تابع دقیقاً  $N \cdot M$  بار در هر نمونه، پس از فراخوانی اولیه‌ی تابع `init` فراخوانی می‌شود.



این تابع باید یک جفت عدد صحیح  $(i, j)$  را برگرداند که در آن  $i$  مختصات سطر و  $j$  مختصات ستون خانه‌ای است که کاشی بالا سمت چپ این بلوک باید در آن قرار گیرد. مقادیر  $i$  و  $j$  باید زوج باشند و ناحیه‌ی  $2 \times 2$  که توسط هر بلوک پوشش داده می‌شود نباید با هیچ بلوک قرار داده شده‌ی دیگری همپوشانی داشته باشد.

اگر تابع `receive_block` جفتی را برگرداند که این الزامات را برآورده نکند یا اگر پس از قرار دادن بلوک، یک مربع  $2 \times 2$  از خانه‌ها کاملاً با کاشی‌های سیاه پوشانده شود، ارزیاب بلافاصله برنامه‌ی شما را خاتمه می‌دهد و نتیجه برای نمونه `Output isn't correct` خواهد بود.

رفتار ارزیاب **تطبیقی** نیست. این بدان معناست که مجموعه بلوک‌هایی که بارچین به چاروس می‌دهد، قبل از فراخوانی تابع `init` مشخص است.

## محدودیت‌ها

برای هر بلوک، فرض کنید  $S$  تعداد کاشی‌های سیاه در بین چهار کاشی آن باشد. یعنی  $S = TL + TR + BL + BR$ .

- $1 \leq N, M \leq 100$
- $0 \leq S \leq 3$  برای هر بلوک.

## زیرمسئله‌ها

| Subtask | Score | Additional Constraints                                                                                                               |
|---------|-------|--------------------------------------------------------------------------------------------------------------------------------------|
| 1       | 6     | $S = 1$ for every block, and $N = 2$ .                                                                                               |
| 2       | 16    | $S = 3$ for every block. $N = M$ , $N$ is even, and each of the four possible block colorings appears exactly $\frac{N^2}{4}$ times. |
| 3       | 10    | $S = 1$ for every block.                                                                                                             |
| 4       | 29    | $S \leq 2$ for every block.                                                                                                          |
| 5       | 39    | No additional constraints.                                                                                                           |

## مثال

یک بازی با  $N = 1$  و  $M = 2$  را در نظر بگیرید. جدول دارای 2 ردیف و 4 ستون است. ارزیاب ابتدا تابع زیر را صدا می‌زند:

```
init(1, 2)
```

در ابتدا، تمامی خانه‌ها خالی هستند. جدول به این شکل است:

|   | 0 | 1 | 2 | 3 |
|---|---|---|---|---|
| 0 |   |   |   |   |
| 1 |   |   |   |   |

تعداد  $N \cdot M = 2$  بلوک برای قرار دادن وجود دارد. فرض کنید بارچین بلوکی با سه کاشی سیاه و یک کاشی سفید در گوشه بالا سمت راست می‌دهد. نمره‌گذار می‌گوید:

```
receive_block(1, 0, 1, 1)
```

چاروس تصمیم می‌گیرد با برگرداندن مقدار  $(0, 0)$  این بلوک را در سمت چپ جدول قرار دهد.

اکنون جدول به این شکل است:

|   | 0 | 1 | 2 | 3 |
|---|---|---|---|---|
| 0 |   |   |   |   |
| 1 |   |   |   |   |

سپس بارچین بلوک دیگری با سه کاشی سیاه و یک کاشی سفید در گوشه‌ی بالا سمت چپ ارائه می‌دهد:

```
receive_block(0, 1, 1, 1)
```

تنها خانه‌ی باقی مانده با سطر و ستون زوج که می‌تواند به عنوان گوشه‌ی بالا سمت چپ یک بلوک  $2 \times 2$  عمل کند خانه‌ی  $(0, 2)$  است. بنابراین چاروس مقدار  $(0, 2)$  را برمی‌گرداند. جدول به این شکل در می‌آید:

|   | 0 | 1 | 2 | 3 |
|---|---|---|---|---|
| 0 |   |   |   |   |
| 1 |   |   |   |   |

هیچ مربع  $2 \times 2$  ای به طور کامل با کاشی‌های سیاه پوشیده نشده است. بنابراین چاروس با موفقیت تمام بلوک‌ها را بدون اینکه بارچین برنده شود، قرار داده است و بازی را می‌برد.

## ارزیاب نمونه

قالب ورودی:

```
N M
TL[0] TR[0] BL[0] BR[0]
TL[1] TR[1] BL[1] BR[1]
...
TL[NM-1] TR[NM-1] BL[NM-1] BR[NM-1]
```

قالب خروجی:

```
R[0] C[0]
R[1] C[1]
...
R[NM-1] C[NM-1]
```

در این جا  $R[k]$  و  $C[k]$ ، جفت اعداد صحیحی هستند که توسط فراخوانی  $k$  ام به `receive_block` بازگردانده می‌شوند.